

Plan de Implementación — Tracking de Remediación Sólido

Nota de alcance

Este plan traduce los 11 puntos de `Brechas_Tracking_Remediacion.md` (Sección 2) en fases ejecutables. Cada fase indica: qué capas toca (backend/n8n/frontend), el **nivel de gobernanza** (según la matriz de autonomía por dominio — Auth/Seguridad/Auditoría = CRÍTICA, lógica de negocio = MEDIA, CRUD simple = BAJA), los archivos concretos a tocar, los pasos, y cómo verificarlo.

Regla dura de este plan: ninguna fase se implementa sin que el usuario apruebe explícitamente el plan de esa fase primero — es la disciplina ya establecida en este proyecto (ver `docs/20-08-26/`). Las fases con gobernanza CRÍTICA además requieren aprobación explícita del *diseño* antes de escribir una sola línea, no solo del plan general.

Orden: el mismo de `Brechas_Tracking_Remediacion.md` Sección 2 — cada fase depende de que la anterior exista. No saltar fases salvo que se decida explícitamente achicar el alcance.

Disciplina de ejecución por fase (una vez aprobada), la misma que ya usa el equipo en este proyecto: 1. Leer el código actual real antes de tocarlo (no asumir). 2. Escribir el test que falla antes del cambio y pasa después — nunca un test tautológico. 3. Implementar el cambio mínimo que hace pasar ese test. 4. Compilar (`./gradlew compileJava / compileTestJava`), correr el test puntual, después el suite completo (`./gradlew test`). 5. Si toca backend: rebuild + redeploy del contenedor, esperar healthcheck. 6. Si toca n8n: editar `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades (VMP).json`, `docker restart vuln_n8n`. 7. Si toca frontend: `npm run build && npm run test`, rebuild + redeploy. 8. Verificar en vivo (login real, curls contra endpoints reales, o Playwright si es visual) — nunca dar algo por resuelto solo porque compila. 9. Limpiar cualquier dato de prueba generado durante la verificación. 10. Marcar la fase [x] acá abajo, con el detalle de qué se verificó.

Checklist general

- [x] Fase 1 — Auditoría de transiciones + separación ciclo/hallazgo (Componente F + E) — 2026-08-21
- [x] Fase 2 — Exponer `reopenCount` y tasa de recurrencia — 2026-08-21
- [x] Fase 3 — Cierre consciente de cobertura (Componente A) — 2026-08-21
- [x] Fase 4 — MTTR nominal vs. verificado + aging + tiempo por criticidad — 2026-08-21
- [x] Fase 5 — Estados VEX (mitigada / no aplica / riesgo aceptado) — 2026-08-21
- [x] Fase 6 — Persistir variables de prioridad + campo de plazo/SLA — 2026-08-21
- [x] Fase 7 — Escala de evidencia + captura de respaldo — 2026-08-21
- [x] Fase 8 — GHSA estructurado + verificación real por rango de versión — 2026-08-21
- [x] Fase 9 — Normalizar `assignedTo` a relación con `User` — 2026-08-21
- [x] Fase 10 (opcional) — Campo `exposed` en `Asset` — 2026-08-21

Fase 1 — Auditoría de transiciones + separación ciclo/hallazgo

Componente F + E · Gobernanza: ● **CRÍTICA** (registro de auditoría — según la matriz de autonomía, esto es análisis y diseño primero, sin escribir código hasta aprobación explícita del diseño de datos, no solo del plan).

Por qué primero: sin esto, ninguna de las fases siguientes queda auditable retroactivamente.

Qué construir: - Entidad nueva de transición (`VulnerabilityStateTransition` o similar): `id`, `cycle_id` (o `finding_id` si no se separa el ciclo todavía), `from_state`, `to_state`, `occurred_at`, `actor_id`, `actor_type` (`HUMANO` / `ESCANER` / `POLITICA`), `trigger`, `evidence_ref` (nullable por ahora, se llena en Fase 7), `comment`. - Decisión de diseño a resolver **antes de codear** (esto es lo que corresponde aprobar explícitamente): ¿se separa ya "ciclo de remediación" de "hallazgo persistente" (dos entidades, `Finding` + `RemediationCycle`), o se agrega la tabla de transiciones sobre el modelo actual y se pospone la separación de ciclos? El documento recomienda hacerlas juntas porque separar después obliga a rehacer las consultas de métricas ya construidas — pero es más trabajo de entrada. Decidir esto es la aprobación de diseño que bloquea el resto. - Poblar la tabla nueva en dos puntos: `VulnerabilityAuditService.updateStatus()` (transición manual del Kanban) y en cada rama de `applyLifecycle()` (transiciones automáticas: reapertura, cierre por ausencia).

Archivos: `backend/src/main/java/.../model/` (entidad nueva + posible refactor de `AssetVulnerability`), `VulnerabilityAuditService.java`, nueva migración de base de datos, nuevo repositorio.

Tests: cada transición (manual y automática) debe dejar una fila nueva sin tocar las anteriores; reabrir un caso no debe borrar el historial del ciclo previo.

✅ Completado — 2026-08-21

Decisiones de diseño aprobadas explícitamente antes de codear: (1) separar ya "ciclo de remediación" (`RemediationCycle`) de "hallazgo persistente" (`AssetVulnerability`, sin renombrar); (2) al migrar los casos existentes, backfillar un `RemediationCycle` por fila con datos reales, **sin** fabricar transiciones de arranque — el trail de auditoría empieza vacío para los casos viejos.

Construido: - `RemediationCycle.java` / `StateTransition.java` (modelo), `RemediationCycleRepository.java` / `StateTransitionRepository.java`. - `init/19-remediation-cycles-and-transitions.sql` — tablas nuevas + backfill (131 ciclos, 0 transiciones fabricadas). - `VulnerabilityAuditService.updateStatus()`: escribe `StateTransition` (actor `HUMANO`, trigger `KANBAN_MANUAL`) y cierra el ciclo vigente si el nuevo estado es RESUELTA. - `VulnerabilityAuditService.applyLifecycle()`: crea ciclo #1 + transición (`ESCANER`, `ESCANEO_N8N`) en detección nueva; crea ciclo nuevo + transición en reapertura (`ESCANER`, `REAPERTURA_ESCANEO`); cierra el ciclo vigente + transición (`POLITICA`, `CIERRE_POR_AUSENCIA`) en cierre automático por ausencia.

Tests: 7 tests nuevos/actualizados en `VulnerabilityAuditServiceTest.java` (creación, reapertura, cierre manual con cierre de ciclo, cierre por ausencia). Suite completo: 166 tests, 0 fallas.

Verificado en vivo: rebuild + redeploy de `gef_backend`, healthcheck OK. Login real como `analyst.demo`, movida la vulnerabilidad `CVE-2026-42036` (id=25) de DETECTADA → EN_ANALISIS → DETECTADA vía `/api/vulnerabilities/{id}/status`. Confirmado por `SELECT` directo contra Postgres: 2 filas en `state_transitions`, ninguna sobrescrita, `actor_id=13` (`analyst.demo`), `actor_type=HUMANO` en ambas.

Pendiente de verificación en vivo (cubierto solo por test unitario, no por un escaneo real): las ramas automáticas de `applyLifecycle()` (reapertura y cierre por ausencia) requieren correr un escaneo real de n8n para confirmarse end-to-end — no se disparó ninguno en esta sesión para no generar actividad de escaneo de prueba sobre datos reales.

Fase 2 — Exponer `reopenCount` y tasa de recurrencia

Componente E · Gobernanza: 🟢 **BAJA** (CRUD/lectura de un dato que ya existe).

Qué hacer: - Quitar `@Mapping(target = "reopenCount", ignore = true)` de `VulnerabilityAuditMapper.java`. - Agregar `reopenCount` al DTO correspondiente. - Nueva query en `DashboardService` / `AssetVulnerabilityRepository` para tasa de recurrencia (proporción de casos cerrados que reabrieron al menos una vez). - Frontend: tarjeta o indicador nuevo en `Dashboard.tsx`.

Tests: un caso reabierto 2 veces debe reflejarse en el DTO y en la tasa calculada.

✅ Completado — 2026-08-21

Corrección de diseño: no había ningún `@Mapping(ignore = true)` que quitar — `reopenCount` directamente no existía en `VulnerabilityAuditDTO.Response`. Se agregó el campo (`MapStruct` lo mapea automáticamente por nombre) y `reopenedCasesCount` / `recurrenceRate` en `DashboardStatsDTO`, con nuevas queries `countByReopenCountGreaterThan(0)` (global y scopeada) en `AssetVulnerabilityRepository`. Frontend: tarjeta "Tasa de recurrencia" en `Dashboard.tsx`.

Tests: 4 nuevos (mapper + 2 en `DashboardServiceTest`, incluido el caso de 0 vulnerabilidades para no dividir por cero). Suite completo backend: verde. Frontend: build + test OK.

Verificado en vivo: rebuild+redeploy de backend y frontend. `GET /api/dashboard/stats` real devuelve `reopenedCasesCount=115`, `recurrenceRate=0.878` sobre 131 hallazgos — verificado contra un `SELECT reopen_count, COUNT(*) ... GROUP BY` directo en Postgres antes de aceptar el número (115 de 131 filas tienen `reopen_count > 0`, coincide exacto). El valor alto es real, no un bug: refleja la actividad de re-escaneo repetido de este entorno de desarrollo.

Fase 3 — Cierre consciente de cobertura

Componente A · Gobernanza: 🟡 **MEDIA** (lógica de negocio central del ciclo de vida — implementar con checkpoint, surfacear la decisión de diseño antes de aplicarla).

Qué hacer (parte barata, hacer primero): - En `VulnerabilityAuditService.applyLifecycle()`, antes de marcar `RESOLVED` por ausencia: consultar `ScanReport.systemStatus` del escaneo actual; si contiene la marca de parcial ("⚠️ PARCIAL..."), no cerrar por ausencia — dejar el hallazgo en su estado, marcado de alguna forma como "no evaluado esta corrida" en vez de "resuelto".

Qué hacer (segunda vuelta, más cara — opcional en esta fase): - `ScanReport`: agregar `assetsInScope` / `assetsAnalyzed`. - n8n: `Persist_Scan_Statistics` debe empezar a mandar esos conteos en el payload del webhook.

Archivos: `VulnerabilityAuditService.java` (líneas ~337-348), `ScanReport.java`, posible migración, `workflows/Pipeline de Gestión Automatizada de Vulnerabilidades (VMP).json` (solo si se hace la segunda vuelta).

Tests: un escaneo con `systemStatus` parcial no debe cerrar hallazgos por ausencia; uno completo sí debe seguir cerrando como hoy.

✅ Completado — 2026-08-21 (implementación distinta a la propuesta original)

Corrección importante encontrada al revisar el workflow de n8n real (`Compute_Audit_Metrics`): el string `systemStatus` es **ambiguo** — si el escaneo tiene items no evaluados **y además** alguna criticidad detectada, n8n muestra "⚠️ ACCIÓN REQUERIDA", no "⚠️ PARCIAL...". Buscar el texto "PARCIAL" habría dejado pasar exactamente el caso más importante (cobertura incompleta + hallazgos críticos reales).

Fix implementado en su lugar: `ScanReport.totalDetected`, `.audited` y `.ignored` ya llegan completos desde n8n en cada escaneo (no requiere ningún cambio en el pipeline). Si `audited + ignored < totalDetected`, hay ítems sin evaluar — cobertura incompleta, sin depender de ningún texto. `applyLifecycle()` ahora corta el loop de cierre-por-ausencia completo (no solo un caso puntual) cuando esto ocurre, y lo loguea. Escaneos legacy sin estos tres campos poblados se tratan como cobertura completa (no se puede afirmar lo contrario sin el dato).

No se hizo la segunda vuelta (`assetsInScope` / `assetsAnalyzed` explícitos) — innecesaria: el fix barato ya resultó más preciso que el propuesto originalmente.

Tests: 2 nuevos (cobertura incompleta no cierra; cobertura completa sí cierra como antes). Suite completo: 168 tests, 0 fallas.

Verificado: rebuild+redeploy de `gef_backend`, healthcheck OK, smoke test de `/api/vulnerabilities` con login real. **No se verificó en vivo el caso de cobertura incompleta en sí** (requeriría forzar una falla real de GitHub Advisory en n8n) — cubierto solo por test unitario.

Fase 4 — MTTR nominal vs. verificado + KPIs mecánicos

Componente B/métricas · Gobernanza: 🟢 **BAJA** / 🟡 **MEDIA** (cambios de queries y dashboard; sin tocar el modelo de estados).

Qué hacer: - Renombrar/separar en `DashboardStatsDTO`: `mttrDeclaredDays` (lo que hoy es `mttrDays`) y dejar preparado `mttrVerifiedDays` para cuando exista verificación real (Fase 8). - Nueva query de **aging**: bucketizar `firstDetectedAt` de lo abierto en 0-30/31-60/61-90/+90 días. - Nueva query de **tiempo promedio por criticidad**: el mismo cálculo de `findAverageMttrDays()`, agrupado por `priority`. - Frontend: `Dashboard.tsx` — nuevo gráfico de aging, tarjeta de MTTR desdoblada, texto que declare el ancla ("desde primera detección hasta marca manual").

Archivos: `DashboardService.java`, `AssetVulnerabilityRepository.java`, `DashboardStatsDTO.java`, `Dashboard.tsx`.

Tests: casos con distintas antigüedades deben caer en el bucket correcto; el promedio por criticidad debe diferir entre CRITICAL y LOW en un dataset de prueba armado a propósito.

✅ Completado — 2026-08-21

Construido: `mttrDays` renombrado a `mttrDeclaredDays` en todo el proyecto (backend + frontend, sin dejar el campo viejo como alias — se controla el mismo código en ambas puntas en esta sesión); `mttrVerifiedDays` agregado, siempre `null` hasta la Fase 8. Nuevas queries `countAgingBuckets()` y `findAverageMttrDeclaredDaysByPriority()` (globales y scopeadas) en `AssetVulnerabilityRepository`. `ReportPdfService` actualizado con la etiqueta "MTTR declarado". Frontend: tarjeta MTTR renombrada, tarjeta nueva de tasa de recurrencia, dos gráficos nuevos (aging, MTTR por criticidad).

Tests: 3 nuevos + renombrado de mocks en 4 tests existentes (`DashboardServiceTest`, `ReportPdfServiceTest`). Suite completo backend: verde. Frontend: build + test OK.

Verificado en vivo con captura de pantalla real (Playwright, login como `analyst.demo`): las 3 tarjetas y 2 gráficos nuevos renderizan correctamente — "MTTR declarado" (10.7 d), "Tasa de recurrencia" (87.8%), aging (bucket 0-30 con 115 casos), MTTR por criticidad con las 4 prioridades y sus colores correctos.

Fase 5 — Estados VEX (mitigada / no aplica / riesgo aceptado)

Componente B · Gobernanza: 🟡 **MEDIA** (cambia la máquina de estados — surfacear las reglas de transición nuevas antes de implementar).

Qué hacer: - Ampliar el modelo de estados: agregar un campo `outcome` (o extender `triageStatus`) con `MITIGADA`, `NO_APLICA`, `RIESGO_ACEPTADO`, además de las reglas de transición que correspondan desde cada estado existente. - Campos nuevos: `mitigationControl` (texto, obligatorio si `MITIGADA`), `justification` (texto, obligatorio si `NO_APLICA`), `acceptedBy` + `riskAcceptedUntil` (fecha, **obligatoria** si `RIESGO_ACEPTADO` — sin esto el antipatrón de aceptación perpetua vuelve a aparecer). - Frontend: Kanban — nuevas opciones en el modal de gestión (no necesariamente nuevas columnas visuales, puede ser un selector dentro de la tarjeta). - Validación: rechazar la transición si falta el campo obligatorio correspondiente.

Archivos: `AssetVulnerability.java`, `VulnerabilityAuditService.java` (reglas de transición), migración, `Kanban.tsx`.

Tests: intentar marcar `RIESGO_ACEPTADO` sin fecha de caducidad debe fallar; marcar `NO_APLICA` sin justificación debe fallar.

✓ Completado — 2026-08-21

Diseño: `outcome` como campo separado (no se extendió `triageStatus`) — solo tiene sentido cuando `triageStatus = RESUELTA`, describe *cómo* se cerró. `null` = cierre plano (compatibilidad total con lo que ya existía). Campos de respaldo (`mitigationControl` / `justification` / `acceptedBy` + `riskAcceptedUntil`) se limpian automáticamente al mover el caso lejos de `RESUELTA` — mismo criterio que ya aplicaba `resolvedAt`.

Tests: 7 nuevos (4 de rechazo, 3 de aplicación correcta + limpieza al reabrir). Suite completo backend: 174 tests, 0 fallas. Frontend build+test OK.

Verificado en vivo: flujo completo real vía `PATCH /api/vulnerabilities/{id}/status` — cierre con `RIESGO_ACEPTADO` completo (200, persistido correcto en Postgres); intento sin `riskAcceptedUntil` rechazado con 409 y **sin tocar la fila** (confirmado por SELECT); reversión a `EN_ANALISIS` limpia los 4 campos de `outcome` (confirmado por SELECT). Screenshot real del modal del Kanban con Playwright: el selector "Cómo se cerró" y los campos condicionales de "Riesgo aceptado" renderizan correctamente.

Fase 6 — Persistir variables de prioridad + campo de plazo/SLA

Componente C · Gobernanza: ● **MEDIA** (toca n8n y backend; es agregar transparencia a un cálculo que ya existe, no rediseñarlo).

Qué hacer: - n8n: `Risk_Assessment_Engine` debe incluir en su salida el desglose de variables (score CVSS, `env_criticality` usado, si hubo match de KEV, si es zero-day) — hoy solo emite `priority` y `action_plan` como texto. - n8n: el payload que llega al webhook (o el `INSERT` de `Persist_Audit_Findings`, según dónde se decida guardarlo) debe llevar ese desglose. - Backend: nuevo campo (JSON o columnas propias) `priorityVariables` en `AssetVulnerability` / `VulnerabilityAudit`. - Backend: nuevo campo `dueDate`, calculado a partir de esas variables con un criterio documentado en el propio código (comentario o constante nombrada, no un número mágico).

Archivos: workflow de n8n, `VulnerabilityAudit.java`, `AssetVulnerability.java`, `VulnerabilityAuditService.java`, migración.

Tests: un caso `CRITICAL` con `KEV=true` debe tener un `dueDate` más corto que uno `CRITICAL` sin KEV, verificable con datos de prueba controlados.

✓ Completado — 2026-08-21

n8n editado directamente (con backup previo del workflow): `Risk_Assessment_Engine` arma `priority_variables` (JSON: `cvss_score`, `env_criticality`, `priority_base`, `is_cisa_kev`, `is_zero_day`, `priority_final`); `Persist_Audit_Findings` lo inserta como columna `jsonb`. Reinicio de `vuln_n8n` para tomar el cambio.

Backend: columna `priority_variables` (jsonb) en `vulnerabilities_audit` y `asset_vulnerabilities`; `due_date` en `asset_vulnerabilities`, calculado por `VulnerabilityAuditService.calculateDueDate()` a partir de `priority` (`CRITICAL=3d`, `HIGH=7d`, `MEDIUM=30d`, `LOW=90d` — el mismo ejemplo simple de industria que cita el propio marco en su Sección 13.3, no el modelo formal de la BOD 26-04 que sigue fuera de alcance). Se recalcula en cada escaneo que toque el hallazgo — no queda "congelado" si la prioridad cambia.

Tests: 3 nuevos (creación con variables, recálculo de `dueDate` al escalar prioridad, mapper). Suite completo: 183 tests, 0 fallas.

Verificado en vivo con un escaneo real disparado por la API (no simulado): `POST /api/software-components/3/scan` → `SELECT` directo en Postgres confirmó `priority_variables` real generado por n8n en `vulnerabilities_audit` (`scan_id=257`) y propagado a `asset_vulnerabilities` con `due_date` correcto (`HIGH` → detección+7 días, `MEDIUM` → +30 días, ancla en `first_detected_at`, no en la última re-detección). Confirmado también vía `GET /api/vulnerabilities/{id}` y visualmente en el modal del Kanban ("Vence: 25/8/2026").

Fase 7 — Escala de evidencia + captura de respaldo

Componente B/D · Gobernanza: ● **MEDIA**

Qué hacer: - Campo `evidenceLevel` (enum E0-E6 o versión simplificada de 3-4 niveles) en la transición de cierre. - Campo `evidenceRef` (texto/URL) para linkear el respaldo real (reporte de escáner, commit, PR) — se completa el `evidence_ref` que quedó nullable en la Fase 1. - Frontend: input de texto/URL en el modal de cierre del Kanban.

Archivos: entidad de transición (Fase 1), `VulnerabilityAuditService.java`, `Kanban.tsx`.

Tests: un cierre sin nivel de evidencia debe caer en `E0` por defecto, no quedar `null`.

✓ **Completado — 2026-08-21**

Escala completa E0-E6 (no la versión simplificada) en `StateTransition.evidenceLevel` + `evidenceRef` ya existente desde la Fase 1. Solo se completa en cierres **manuales** del Kanban — las transiciones automáticas (ESCANER/POLITICA) quedan `null` a propósito, porque la escala describe una *declaración* de evidencia, y esas transiciones no son eso.

Tests: 3 nuevos (default E0, valores explícitos persistidos, no se completa fuera de un cierre). Suite completo: 186 tests, 0 fallas. Frontend build+test OK.

Verificado en vivo: cierre real con `evidenceLevel=E4` + `evidenceRef` (URL de PR) — confirmado por SELECT en Postgres; cierre real sin especificar nivel — confirmado `E0` en la fila; transiciones no-cierre (DETECTADA ↔ EN_ANALISIS) confirmado `evidence_level` vacío. Screenshot del modal del Kanban con los campos "Nivel de evidencia" y "Respaldo" renderizando correctamente.

Fase 8 — GHSA estructurado + verificación real por rango de versión

Componente D · Gobernanza: ● **MEDIA**/● **alta sensibilidad técnica** (es el cambio que más afecta el comportamiento de cierre automático — probar exhaustivamente antes de desplegar).

La fase técnicamente más importante de todo el plan — es la que efectivamente mueve al sistema de nivel 2 a nivel 3 de madurez de verificación.

Qué hacer: - n8n: `Fetch_GitHub_Advisories` debe pedir y `Risk_Assessment_Engine` debe parsear: rangos completos (`introduced / fixed / last_affected`), `withdrawn_at`, estado `reviewed / unreviewed`. - n8n: `Persist_Audit_Findings` (el INSERT directo a `vulnerabilities_audit`) debe incluir esos campos nuevos. - Backend: `GhsaAdvisoryCache` — nuevo modelo de rango estructurado, posible tabla separada si un advisory tiene múltiples rangos por paquete. - Backend: nueva lógica de comparación en `VulnerabilityAuditService` — para cada hallazgo abierto, comparar la versión actual del componente contra el rango vulnerable del advisory (pseudocódigo de la Sección 10.2 del marco), en vez de (o además de) inferir por ausencia. Advisories `unreviewed` o con rango placeholder (`>= 0`) deben tratarse con confianza más baja, no como verificación fuerte. - Advisories `withdrawn` no deben mantener abierto un hallazgo basado en ellos.

Archivos: workflow de n8n (dos nodos), `GhsaAdvisoryCache.java`, `GhsaAdvisoryService.java`, `VulnerabilityAuditService.java`, migración.

Tests (los más importantes de todo el plan): advisory con rango simple → cierre verificado correcto; advisory con múltiples rangos → todos evaluados; advisory `withdrawn` → no debe sostener un hallazgo abierto; advisory `unreviewed` con rango placeholder → tratarse con confianza baja, no cerrar en automático sin revisión.

✓ **Completado — 2026-08-21**

Desviación de diseño importante, encontrada al revisar el pipeline real: la API REST de GitHub Advisories no devuelve rangos en formato OSV (`introduced / fixed / last_affected`) — eso es específico del formato OSV (`api.osv.dev`), una fuente distinta. Lo que GitHub sí devuelve, y ya estaba disponible sin cambiar la petición HTTP, es `vulnerable_version_range` (string, ej. `">= 1.0.0, < 2.3.4"`), `type` (`reviewed / unreviewed / malware`) y `withdrawn_at`. Se modeló con estos campos reales en vez de inventar una estructura OSV que la fuente no provee.

Hallazgo de arquitectura de n8n: el nodo `Extract_Advisory_Items` (`splitOut` sobre `vulnerabilities`) descartaba `type` y `withdrawn_at` antes de que llegaran a `Risk_Assessment_Engine` (`fieldsToInclude` solo listaba `ghsa_id`, `cve_id`, `cvss_severities`) — hubo que agregarlos ahí, no solo en el nodo de cálculo.

n8n (3 nodos editados, con backup previo): - `Extract_Advisory_Items`: agrega `type`, `withdrawn_at` a `fieldsToInclude`. - `Risk_Assessment_Engine`: descarta el item si `withdrawn_at` está presente (mismo criterio que "ya no es vulnerable"); agrega `advisory_type` y `vulnerable_version_range` a la salida. - `Persist_Audit_Findings`: inserta las 2 columnas nuevas.

Backend: `advisory_type / vulnerable_version_range` en `vulnerabilities_audit` y `asset_vulnerabilities` (denormalizado, mismo patrón que `patched_version`). `GhsaAdvisoryCache / GhsaAdvisoryService` (el otro lado del sistema — la cache de presentación) enriquecidos con `reviewed / withdrawnAt`, parseados del `type / withdrawn_at` de la API de GitHub.

Decisión sobre "no cerrar en automático sin revisión": en vez de bloquear el cierre automático para advisories `unreviewed` /rango placeholder (lo que hubiera dejado la mayoría de los cierres reales sin cerrar nunca), se optó por **etiquetar la confianza** del cierre: `evidenceLevel = E4` (inventario posterior con rango real, advisory revisado) o `E2` (advisory no revisado o rango placeholder `>= 0`) en la propia `StateTransition` del cierre por ausencia — reutilizando la escala de la Fase 7. Esto cumple el espíritu del marco (Sección 7.2: no tratar como verificación fuerte lo que no lo es) sin volver el sistema disruptivo.

Tests: 5 nuevos (creación con `advisory_type/rango`, cierre E4 con advisory revisado+rango real, cierre E2 con advisory no revisado, cierre E2 con rango placeholder, mapper). Suite completo: 193 tests, 0 fallas.

Verificado en vivo con dos escaneos reales disparados por la API: `SELECT` directo en Postgres confirmó `advisory_type='REVIEWED'` y `vulnerable_version_range` reales (ej. `">= 1.0.0, < 1.15.1"`) generados por n8n. Confirmado también vía `GET /api/vulnerabilities/{id}`.

Pendiente de verificación en vivo (cubierto solo por test unitario): el filtro de advisories `withdrawn` en n8n y el etiquetado de confianza E2/E4 en un cierre automático real — ningún escaneo de esta sesión generó naturalmente un cierre por ausencia ni encontró un advisory retirado, y no hay forma de forzar cualquiera de los dos sin manipular datos de GitHub.

Fase 9 — Normalizar `assignedTo` a relación con `User`

Componente misceláneo · Gobernanza: ● **BAJA** (la UI ya restringe a usuarios reales; es solo integridad referencial de esquema).

Qué hacer: - Migración: agregar `assigned_to_user_id` (FK a `users`), backfill desde el `assignedTo` (texto) existente por match de username, deprecir la columna vieja (no borrar de entrada — dejarla en paralelo una versión antes de eliminarla). - Actualizar servicio/DTO para usar la FK. - Query nueva de "vulnerabilidades por responsable" ahora que el dato es confiable (cierra el KPI #8 de la Sección 4 de Brechas).

Archivos: `AssetVulnerability.java`, migración, `DashboardService.java` (nuevo query).

Tests: el backfill no debe perder ningún caso ya asignado; un `assignedTo` que no matchea ningún usuario real debe quedar explícitamente `null`, no inventar un usuario.

✓ Completado — 2026-08-21

`assigned_to_user_id` agregado en paralelo a `assigned_to` (texto, no se borró). Backfill real: 6 filas matchearon un username existente, el resto (mayormente "SIN_ASIGNAR") quedó `NULL` — correcto, no se inventó ningún usuario. `updateStatus()` ahora resuelve la FK en cada asignación (`userRepository.findByUsername(...)`, `orElse(null)`). Nueva query `countOpenGroupedByAssignedUser()` (global y scopeada) cierra el KPI #8 de la Sección 4 de Brechas ("vulnerabilidades por responsable"). Frontend: gráfico nuevo "Abiertas por responsable" en el Dashboard.

Tests: 6 nuevos (resuelve FK con usuario real, deja null sin match, no toca el repo si no viene `assignedTo`, KPI por responsable). Suite completo: 200 tests, 0 fallas. Frontend build+test OK.

Verificado en vivo: asignación real vía `PATCH /api/vulnerabilities/{id}/status` con `assignedTo=analyst.demo` — confirmado por `SELECT` en Postgres (`assigned_to_user_id=13`, join a `users` resuelve `analyst.demo`); `GET /api/dashboard/stats` refleja el conteo real por responsable. Screenshot del Dashboard con el gráfico nuevo poblado con datos reales (`auditor.demo`, `analyst.demo`).

Fase 10 (opcional) — Campo `exposed` en `Asset`

Gobernanza: ● **BAJA**. Solo si, después de la Fase 6, se decide que la criticidad de entorno no alcanza y hace falta distinguir específicamente "alcanzable desde internet". No bloquea ninguna otra fase — puede hacerse en cualquier momento o no hacerse.

✓ Completado — 2026-08-21 (implementada pese a ser opcional, a pedido explícito de completar todas las fases)

Construido: `exposed` (Boolean, nullable) en `Asset` — declarado a mano, `null` = no declarado (no se asume "no expuesto"). Migración, DTO, mapper (auto-mapeado por MapStruct), checkbox en el formulario de Activos del frontend. No se integró al cálculo de prioridad de n8n (eso es un paso aparte, explícitamente fuera de esta fase).

Tests: 2 nuevos (mapper `toResponse` / `toEntity`). Suite completo backend: 202 tests, 0 fallas. Frontend build+test OK.

Incidente durante la verificación en vivo, y cómo se resolvió: al probar el endpoint de activos con un `PUT` manual que solo mandaba `name / assetType / exposed` , se perdió el `environment_id` del activo real `id=20` ("Express API Gateway") — el endpoint hace reemplazo completo, no merge parcial (comportamiento preexistente, no introducido por esta fase). No había forma de reconstruir el valor perdido desde los datos existentes (ni en `scan_reports` ni en otra tabla) — se le preguntó al usuario, que confirmó "Desarrollo", y se restauró. Confirmado además que el formulario real del frontend no tiene este problema: `openEditAsset()` ya carga el estado completo (incluido `environmentId`) antes de guardar, así que un usuario real nunca dispararía esta pérdida de datos — solo ocurrió por probar el endpoint crudo con un payload incompleto.

Verificado en vivo por la UI real (no por API cruda, después del incidente): login real, editar el activo restaurado, tildar "Expuesto a internet", guardar — confirmado por `SELECT` en Postgres que `environment_id` se mantuvo intacto y `exposed=true` se guardó correctamente. Revertido a `NULL` (no declarado) al final, estado original.

Fuera de este plan

La Sección 10 de `Brechas_Tracking_Remediacion.md` (seguridad de la plataforma misma — falta de log de eventos de autenticación, sin protección contra fuerza bruta) es un dominio adyacente, no parte del tracking de remediación. Si se quiere abordar, conviene un plan aparte — no mezclarlo con las fases de arriba para no confundir gobernanza (ese dominio sí es CRÍTICO en el sentido estricto de Auth/Seguridad de la matriz de autonomía).

Caja Negra (escaneo externo de puertos/servicios) no está en este plan. Está descartada explícitamente en la Sección 11 de `Brechas_Tracking_Remediacion.md`: requeriría infraestructura de escaneo externo (Nessus/OpenVAS/ `nmap`) — un proyecto de infraestructura en sí mismo, no una tarea incremental — y agregaría una segunda fuente de detección, lo que activaría el problema de deduplicación multi-fuente (también fuera de alcance). No hace falta para llegar a nivel 3 de madurez de verificación. La Fase 10 de este plan (campo `exposed` manual) toca el mismo tema de exposición del activo, pero es solo un dato que carga a mano el analista — no un mecanismo de descubrimiento real, y no requiere ninguna de la infraestructura de Caja Negra.